EUROPEAN UNIVERSITY OF LEFKE

Faculty of Engineering

Department of Software Engineering

COMP XX5

Principles of Digital Image Processing

Term Project:

Custom Image Compressor

Submitted to Dr. **Cem Kalyoncu**

Owolabi Joseph AyOluwa 22146431

1. Introduction

Image compression is an important process in digital image processing, it is designed to reduce the storage space and transmission bandwidth of images while preserving as much of the original data as possible. Generally compression algorithms fall into two categories: lossy (where some data is permanently discarded, like JPEG) and lossless (where the exact original image can be reconstructed, like PNG).

This project implements a custom approach to compress an image compression, leading to the creation of a new format of file (.ay). The main aim of this project is to show how two traditional methods of compression can work together to optimize compression. This is achieved by using Run-Length Encoding and Huffman Encoding.

2. Problem Definition

High-resolution digital images require large amounts of memory because raw image formats use the same amount of data for all pixels. Detecting spatial redundancy means detecting identical pixels next to each other in the form of a uniform background.

It is necessary to develop an algorithm that will be able to perform the following functions:

1. **Detection of Spatial Redundancy:** Identifying cases when identical colors occur between adjacent pixels (flat colors, graphic elements).
2. **Entropy Detection**: Identify what particular bytes (color codes or byte-lengths) occur more often than others throughout the image and create shortened versions in binary format for these bytes.
3. **Serialization of Binary Code**: Implement a method that can efficiently compress any sequence of binary codes into an ordinary sequence of 8-bit bytes.

3. System Overview

The design of the system is such that it implements an automatic pipeline based on the usage of common C++ functions as well as an image.h for simple file reading/writing.

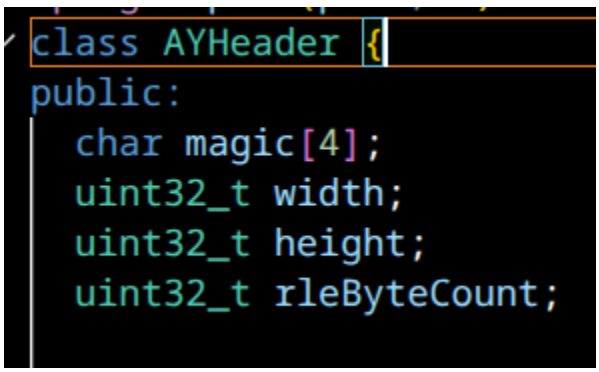
The pipeline consists of the following stages:

- **Encoding:**
    1. **RLE Compression:** The raw RGBA pixels are converted into a flat vector of 5 byte packets (Run Length + Red + Green + Blue + Alpha).
    2. **Frequency Analysis:** Frequency count of each byte (0 – 255) in the RLE payload is performed.
    3. **Construction of Huffman Tree:** The min heap data structure is employed for building a Huffman tree, while a dictionary is built using binary code lengths that vary depending on frequency.
    4. **Packing of Bits:** Four bytes of magic numbers (OWOZ), followed by dimensions, frequencies, and Huffman coded bits are packed into a binary file (.ay).
- **Decoding:**
    1. **Header Verification:** 4-byte magic number is checked for verifying the integrity of data; original width, height, and RLE bytes count are retrieved.
    2. **Tree Construction:** Same Huffman Tree as that used while encoding is constructed based on the frequency table.
    3. **Bit Unpacker:** The binary information will be processed bit by bit to go through the Huffman tree and get the flat RLE information.
    4. **RLE Decompression:** The RLE packets will be converted into RGBA pixels.

4. Method and Explanation

4. 1 The Custom Header (AYHeader)

Reasoning: In order to decode a custom binary file, the program will require a clear understanding of the way the file is structured.



- **char magic[4]:** A magic number (OWOZ) which is 4 bytes long guarantees that the program will not decode a random file, while the size will guarantee proper memory alignment.
- **rleByteCount:** Without it, there would be no way to prevent the program from decoding the "padding bits" of the last byte, which means extra zeroes.Without it, there would be no way to prevent the program from decoding the "padding bits" of the last byte, which means extra zeroes.
- **#pragma pack(push, 1)**: It guarantees that the compiler will align the memory according to our needs using 1-byte alignment only, thus eliminating its tendency to optimize code by adding hidden padding. This guarantees that there will be no padding, so the actual byte layout of the struct is directly mapped to our binary format. Thus, data corruption will be avoided, and cross-platform serialization is ensured, even with future odd sized variable additions.

4.2 Run-Length Encoding (compressRLE)

The algorithm scans the image horizontally. If the current pixel matches the previous pixel, a counter (runLen) increments (capped at 255 to fit in a single byte). If the color changes or the counter reaches 255, the runLen and the RGBA values are pushed to a vector<uint8_t>.

This solves the problem of spatial redundancy. For instance, A line with 100 solid red pixels, which would require 400 bytes (100 * 4), can be represented using only 5 bytes [100, 255, 0, 0, 255].

4.3. Phase 2: Huffman Coding

Frequency Calculation: The algorithm does not take the idea of "pixel" into account but merely counts the number of times that each particular byte (from 0 to 255) appears in the whole stream.

```
struct CompareNode {
    bool operator()(const unique_ptr<HuffNode> &l,
                    const unique_ptr<HuffNode> &r) {
        if (l->freq != r->freq)
            return l->freq > r->freq;
        return l->byteVal > r->byteVal;
    }
};
```

Binary Tree Construction (buildHuffmanTree and rebuildTreeFromTable): There would be one leaf node for each unique byte and the node will go to the heap. The two nodes with the lowest frequencies are popped from the priority queue, joined into a new node, and then pushed back to the priority queue.

std::unique_ptr is used in order to avoid any memory leak. The tie breakers are required, in the case that two different bytes have the exact same frequency. This ensures that the C++ heap algorithm creates the same binary tree structure during both encoding and decoding.

4.4. Phase 3: Bit-Packing (saveAYFile)

As Huffman codes are variable-length strings of 0's and 1's ("101"), they cannot be directly written using ofstream.

```
for (char bit : code) {
    bitBuffer <<= 1;
    if (bit == '1')
        bitBuffer |= 1;
    if (++bitCount == 8) {
        outFile.write(s: reinterpret_cast<const char *>(&bitBuffer), n: 1);
        bitBuffer = 0;
        bitCount = 0;
    }
}
```

In order to resolve this, an intermediate buffer of type uint8_t named bitBuffer is used. Bits are pushed into this buffer starting from the left.Once eight bits are gathered together, the byte is written out onto the hard drive, and the process continues. No memory is wasted by this method.

4.5. Step 4: Decoding and Expansion (loadAYFile)

The decoder relies on the frequency table in order to reconstruct the tree exactly as it was created. Following that, the decoder continues reading each of the bits one at a time with rleData[i] with i = 7 until i >= 0.

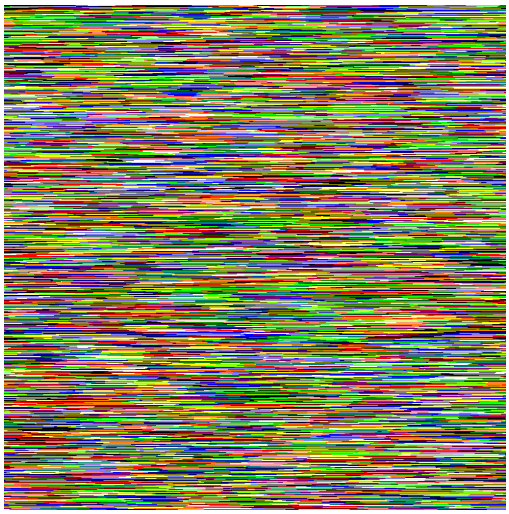
The use of rleData.reserve(header.rleByteCount) is quite effective when it comes to allocating the needed RAM beforehand. The criteria specified through rleData.size() < header.rleByteCount provide the safety of stopping the decode process precisely where it should stop.

5. Results and Limitations

The ultimate compression ratio achieved by the .ay format is strictly governed by the information entropy of the source image. Because the format relies on a pipeline of RLE followed by Huffman coding, its performance swings dramatically depending on whether the input data exhibits geometric predictability or statistical chaos.

Advantages: Low Entropy and Artificial Imagery

The algorithm is particularly effective when handling artificial graphics, uniform color fields, pixel art, interface elements, and pictures with large transparent regions. In such situations, the synergy between RLE and Huffman coding proves highly efficient at squashing both geometric and statistical redundancy.



- **Ideal Test Situation:** In very stringent circumstances with minimalistic input material of low entropy nature, the algorithm succeeded in reducing a 38.5 KB image file to just 20.4 KB (.ay format file), an almost 47% reduction.
- **Why does it work:** Images which contain the same color in all areas will result in long horizontal lines of similar pixels which mathematically are identical. This is the situation where Run Length Encoding really kicks in and reduces numerous bytes to one efficient instruction. The Huffman step then optimizes by giving the most frequently occurring commands the shortest binary codes.

Limitations: High-Entropy and Photographic Imagery

The .ay format is heavily disadvantaged when processing complex, continuous tone photographs, natural noise, or highly detailed textures.

- **The "Lena" Benchmark**: When tasked with compressing the standard 512x512 Lena test image, the format exhibited severe negative

compression (data expansion), ballooning the original 543.3 KB image into a 970.1 KB .ay file (an almost 79% increase in file size).

- The RLE reads data in a straight 1 dimensional line . It struggles fundamentally with anti aliased curves and smooth gradients because, while a gradient looks smooth to the human eye, the actual numerical pixel values are constantly shifting by tiny amounts.

- **Data Inflation:** Because adjacent pixels in photographic images are rarely identical, the RLE phase cannot group them. Instead of compressing the data, RLE causes data inflation. It takes raw, uncompressed pixels and wraps them in RLE control headers (effectively turning a raw 4-byte pixel into a 5-byte or larger packet).
- **Cascading Inefficiency:** This RLE bloat creates a larger, highly chaotic dataset that is then passed to the Huffman phase. Because the data has no clear statistical bias (all colors and packet sizes appear somewhat evenly), the Huffman algorithm cannot generate an efficient tree to compensate for the RLE inflation, resulting in massive file bloat.

Overhead Constraints

Finally, the format is inefficient for extremely small files (such as 16x16 pixel icons). The Huffman coding phase requires a mandatory dictionary (the frequency table) to be embedded directly into the .ay file header so the decoder can reconstruct the data. This table adds roughly 1 KB of fixed overhead to every file. If the original image is only 200 bytes, this mandatory header ensures the resulting .ay file will be over 1.2 KB, regardless of how well the pixel data compresses.

6. Conclusion

The .ay Custom Image Compressor successfully implements a two stage lossless compression pipeline entirely from scratch in standard C++. By treating the RLE output as a raw entropy stream for the Huffman coder, the algorithm dynamically optimizes bit-codes for both colors and run-lengths simultaneously. Memory-safe pointer usage, deterministic heap sorting, and solid bit packing logic ensure that the file format remains highly stable and fault tolerant. This format, however, is not meant to be used as an alternative to predictive two-dimensional formats such as PNG for photographic images.

Appendix: Source code

Github link: https://github.com/Ay-dotcode/Custom-Image-Compressor

```
#include "./image.h"

#include <cstdint>

#include <fstream>

#include <iostream>

#include <memory>

#include <unordered_map>

#include <vector>


using namespace std;


#pragma pack(push, 1)

class AYHeader {

public:

 char magic[4];

 uint32_t width;

 uint32_t height;

 uint32_t rleByteCount;


 AYHeader() : width(0), height(0), rleByteCount(0) {

   magic[0] = 'O';

   magic[1] = 'W';

   magic[2] = 'O';

   magic[3] = 'Z';
```

```
 }
 AYHeader(uint32_t w, uint32_t h, uint32_t rleBytes)
     : width(w), height(h), rleByteCount(rleBytes) {
   magic[0] = 'O';
   magic[1] = 'W';
   magic[2] = 'O';
   magic[3] = 'Z';
 }
 bool isValid() const {
   return (magic[0] == 'O' && magic[1] == 'W' && magic[2] == 'O' &&
           magic[3] == 'Z');
 }
};
#pragma pack(pop)

struct HuffNode {
 uint8_t byteVal;
 uint32_t freq;
 unique_ptr<HuffNode> left;
 unique_ptr<HuffNode> right;

 HuffNode(uint8_t val, uint32_t f)
     : byteVal(val), freq(f), left(nullptr), right(nullptr) {}

 HuffNode(unique_ptr<HuffNode> l, unique_ptr<HuffNode> r)
```

```
      : byteVal(0), freq(l->freq + r->freq), left(std::move(l)),
        right(std::move(r)) {}
};

struct CompareNode {
 bool operator()(const unique_ptr<HuffNode> &l,
                 const unique_ptr<HuffNode> &r) {
   if (l->freq != r->freq)
     return l->freq > r->freq;
   return l->byteVal > r->byteVal;
 }
};

bool saveAYFile(const string &, const ColorImage &);
ColorImage loadAYFile(const string &);
bool isSameColor(const RGBA &, const RGBA &);
vector<uint8_t> compressRLE(const ColorImage &);
ColorImage decompressRLE(const vector<uint8_t> &, uint32_t, uint32_t);
unique_ptr<HuffNode> rebuildTreeFromTable(uint32_t freq[256]);
unique_ptr<HuffNode> buildHuffmanTree(const vector<uint8_t> &);
void generateCodes(HuffNode *, string, unordered_map<uint8_t, string> &);

int main() {
 cout << "\n  .ay Custom Image Compressor\n";
```

```
string inputPng = "input.png";
string outputAy = "data.ay";
string outputPng = "output.png";

cout << "[1/2] Loading " << inputPng << " for compression...\n";
ColorImage I;
I.Load(inputPng);

if (I.GetWidth() == 0) {
  cout << "Error: Could not find or load " << inputPng << endl;
  return 1;
}

if (saveAYFile(outputAy, I)) {
  cout << "      -> Successfully encoded to " << outputAy << "\n";
} else {
  cout << "Error: Failed to save .ay file.\n";
  return 1;
}

cout << "[2/2] Loading " << outputAy << " for decompression...\n";
ColorImage out = loadAYFile(outputAy);

if (out.GetWidth() > 0) {
  out.Save(outputPng);
```

```
    cout << "      -> Successfully reconstructed to " << outputPng << "\n";
  } else {
    cout << "Error: Failed to decode .ay file.\n";
    return 1;
  }

  cout << "\nPipeline Complete!\n";
  return 0;
}

bool saveAYFile(const string &filename, const ColorImage &img) {
  vector<uint8_t> rlePayload = compressRLE(img);
  if (rlePayload.empty())
    return false;

  uint32_t freq[256] = {0};
  for (uint8_t b : rlePayload)
    freq[b]++;

  unique_ptr<HuffNode> root = buildHuffmanTree(rlePayload);
  unordered_map<uint8_t, string> dictionary;
  generateCodes(root.get(), "", dictionary);

  ofstream outFile(filename, ios::binary);
  if (!outFile)
```

```
  return false;


AYHeader header(img.GetWidth(), img.GetHeight(),
                static_cast<uint32_t>(rlePayload.size()));
outFile.write(reinterpret_cast<const char *>(&header), sizeof(AYHeader));
outFile.write(reinterpret_cast<const char *>(freq), sizeof(freq));


uint8_t bitBuffer = 0;
int bitCount = 0;


for (uint8_t byte : rlePayload) {
  const string &code = dictionary[byte];


  for (char bit : code) {
    bitBuffer <<= 1;
    if (bit == '1')
      bitBuffer |= 1;
    if (++bitCount == 8) {
      outFile.write(reinterpret_cast<const char *>(&bitBuffer), 1);
      bitBuffer = 0;
      bitCount = 0;
    }
  }
}
```

```
 if (bitCount > 0) {
   bitBuffer <<= (8 - bitCount);
   outFile.write(reinterpret_cast<const char *>(&bitBuffer), 1);
 }
 return true;
}

ColorImage loadAYFile(const string &filename) {
 ifstream inFile(filename, ios::binary);
 if (!inFile)
   return ColorImage();

 AYHeader header;
 inFile.read(reinterpret_cast<char *>(&header), sizeof(AYHeader));
 if (!header.isValid())
   return ColorImage();

 uint32_t freq[256];
 inFile.read(reinterpret_cast<char *>(freq), sizeof(freq));

 unique_ptr<HuffNode> root = rebuildTreeFromTable(freq);
 if (!root)
   return ColorImage();

 vector<uint8_t> rleData;
```

```
 rleData.reserve(header.rleByteCount);
 HuffNode *currentNode = root.get();
 uint8_t byte;

 while (rleData.size() < header.rleByteCount &&
        inFile.read(reinterpret_cast<char *>(&byte), 1)) {
   for (int i = 7; i >= 0 && rleData.size() < header.rleByteCount; i--) {
     bool bit = (byte >> i) & 1;
     currentNode = bit ? currentNode->right.get() : currentNode->left.get();

     if (!currentNode->left && !currentNode->right) {
       rleData.push_back(currentNode->byteVal);
       currentNode = root.get();
     }
   }
 }
 return decompressRLE(rleData, header.width, header.height);
}

unique_ptr<HuffNode> rebuildTreeFromTable(uint32_t freq[256]) {
 vector<unique_ptr<HuffNode>> heap;
 for (int i = 0; i < 256; i++)
   if (freq[i] > 0)
     heap.push_back(make_unique<HuffNode>(static_cast<uint8_t>(i), freq[i]));
```

```
 if (heap.empty())
   return nullptr;

 if (heap.size() == 1)
   heap.push_back(make_unique<HuffNode>(static_cast<uint8_t>(0), 0u));

 make_heap(heap.begin(), heap.end(), CompareNode());
 while (heap.size() > 1) {
   pop_heap(heap.begin(), heap.end(), CompareNode());
   unique_ptr<HuffNode> l = std::move(heap.back());
   heap.pop_back();
   pop_heap(heap.begin(), heap.end(), CompareNode());
   unique_ptr<HuffNode> r = std::move(heap.back());
   heap.pop_back();
   heap.push_back(make_unique<HuffNode>(std::move(l), std::move(r)));
   push_heap(heap.begin(), heap.end(), CompareNode());
 }
 return std::move(heap.front());
}
unique_ptr<HuffNode> buildHuffmanTree(const vector<uint8_t> &data) {
 uint32_t freq[256] = {0};
 for (uint8_t b : data)
   freq[b]++;
 return rebuildTreeFromTable(freq);
}
```

```
void generateCodes(HuffNode *node, string path,
                   unordered_map<uint8_t, string> &dict) {
 if (!node)
   return;
 if (!node->left && !node->right) {
   dict[node->byteVal] = path;
   return;
 }
 generateCodes(node->left.get(), path + "0", dict);
 generateCodes(node->right.get(), path + "1", dict);
}

bool isSameColor(const RGBA &p1, const RGBA &p2) {
 return (p1.r == p2.r && p1.g == p2.g && p1.b == p2.b && p1.a == p2.a);
}

vector<uint8_t> compressRLE(const ColorImage &img) {
 vector<uint8_t> data;
 int w = img.GetWidth(), h = img.GetHeight();
 if (w == 0 || h == 0)
   return data;

 RGBA currPix = img(0, 0);
 uint8_t runLen = 0;
```

```
 for (int y = 0; y < h; y++) {
   for (int x = 0; x < w; x++) {
     RGBA nextPixel = img(x, y);
     if (isSameColor(currPix, nextPixel) && runLen < 255)
       runLen++;
     else {
       data.push_back(runLen);
       data.push_back(currPix.r);
       data.push_back(currPix.g);
       data.push_back(currPix.b);
       data.push_back(currPix.a);
       currPix = nextPixel;
       runLen = 1;
     }
   }
 }
 data.push_back(runLen);
 data.push_back(currPix.r);
 data.push_back(currPix.g);
 data.push_back(currPix.b);
 data.push_back(currPix.a);
 return data;
}
```

```
ColorImage decompressRLE(const vector<uint8_t> &data, uint32_t w, uint32_t h)
{
 ColorImage img(w, h);
 int pixIdx = 0;
 size_t dataIdx = 0;
 int totPix = static_cast<int>(w * h);

 while (data.size() - dataIdx >= 5 && pixIdx < totPix) {
   uint8_t len = data[dataIdx++];
   uint8_t r = data[dataIdx++];
   uint8_t g = data[dataIdx++];
   uint8_t b = data[dataIdx++];
   uint8_t a = data[dataIdx++];
   RGBA color(r, g, b, a);

   for (int i = 0; i < len && pixIdx < totPix; i++) {
     img(pixIdx % w, pixIdx / w) = color;
     pixIdx++;
   }
 }
 return img;
}
```